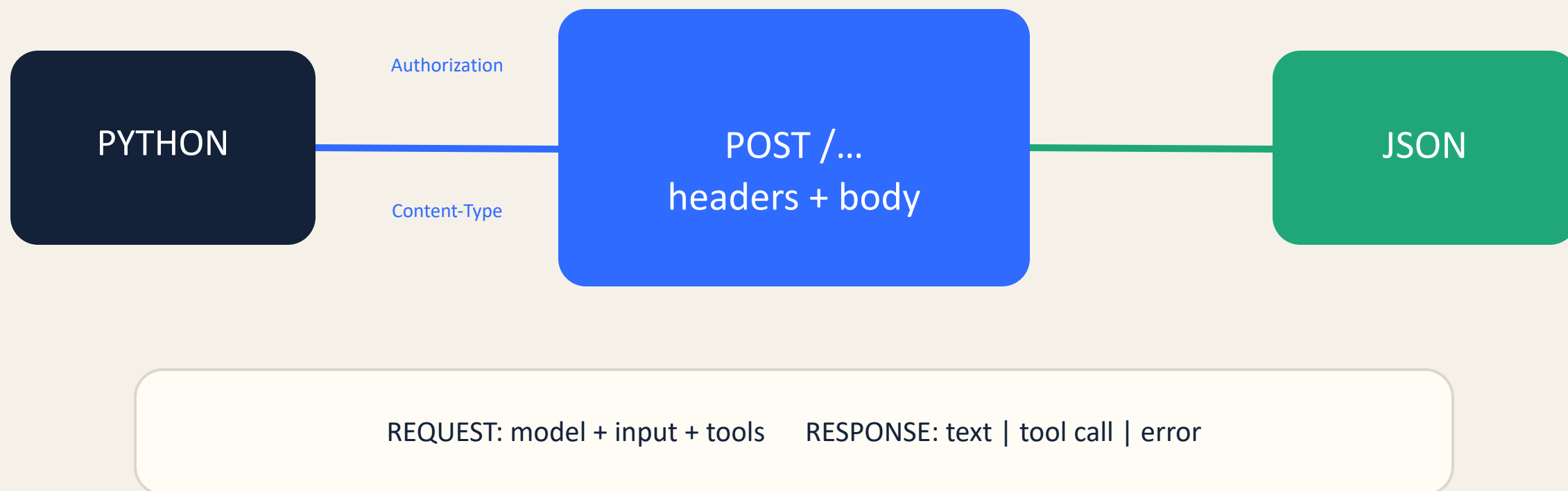


От API-запроса к coding agent



API: по сети всегда уходят HTTP + JSON



Gemini API: четыре режима, один JSON-контракт

Interactions

agentic + multi-turn
рекомендуемый путь

generateContent

один полный ответ
contents → candidates

streamGenerateContent

ответ частями
SSE stream

Live API

двусторонний realtime
WebSocket

<https://ai.google.dev/api/interactions-api-v1?hl=en#>

Одна идея — разные названия полей

API	INPUT	OUTPUT	TOOL REQUEST
Gemini Interactions	input	outputs / steps	function_call
Gemini generateContent	contents.parts	candidates.parts	functionCall
OpenAI Chat Completions	messages	choices[].message	tool_calls
OpenAI Responses	input Items	output Items	function_call

practice/steps/00_api_request.py

СЕЙЧАС

Отправить запросы к api

LLm не видит контекст

calculator.py

tests/

README.md

pyproject.toml

LLM

НЕ ВИДИТ

структуру
код
тесты
правила

Как дать модели только нужные файлы?

practice/steps/01_two_requests_workflow.py

СЕЙЧАС

Запрос №1 выбирает файлы; запрос №2 генерирует правку

Это workflow: маршрут известен до запуска модели



Программа управляет маршрутом; LLM заполняет отдельные шаги

`practice/steps/02_naive_numeric_tools.py`

СЕЙЧАС

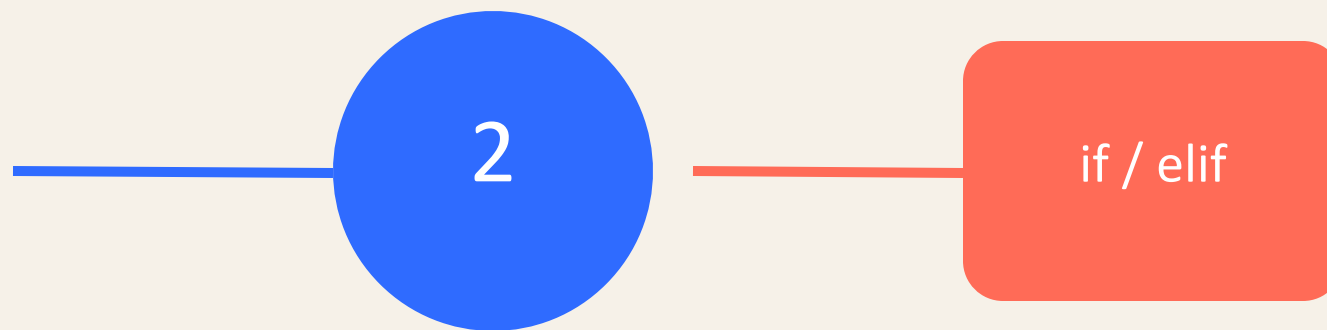
Попросить вернуть только 1 / 2 / 3 и выполнить if / elif

Наивные tools: модель выбирает ветку цифрой

Выбери действие

- 1 список файлов
- 2 открыть файл
- 3 редактировать файл

Верни только цифру



Мы изобрели протокол — без схемы аргументов

Аргументы?



Нет path

Какой файл открыть?



Нет schema

Что считать валидным?



Хрупкий parser

«2.» уже ломает условие



Не расширяется

Четвёртый tool = новый prompt

Нужен стандартный структурированный вызов функции

Function calling = name + JSON arguments + id

function_call

name

read_file

call_id

call_7

arguments

```
{ "path": "calculator.py" }
```

Llm только скажет что хочет это вызвать

practice/steps/03_one_tool_call.py

СЕЙЧАС

Заменить цифру одним настоящим tool call: read_file

Tool declaration — контракт между моделью и кодом

name

read_file

description

Read one UTF-8 file inside repository

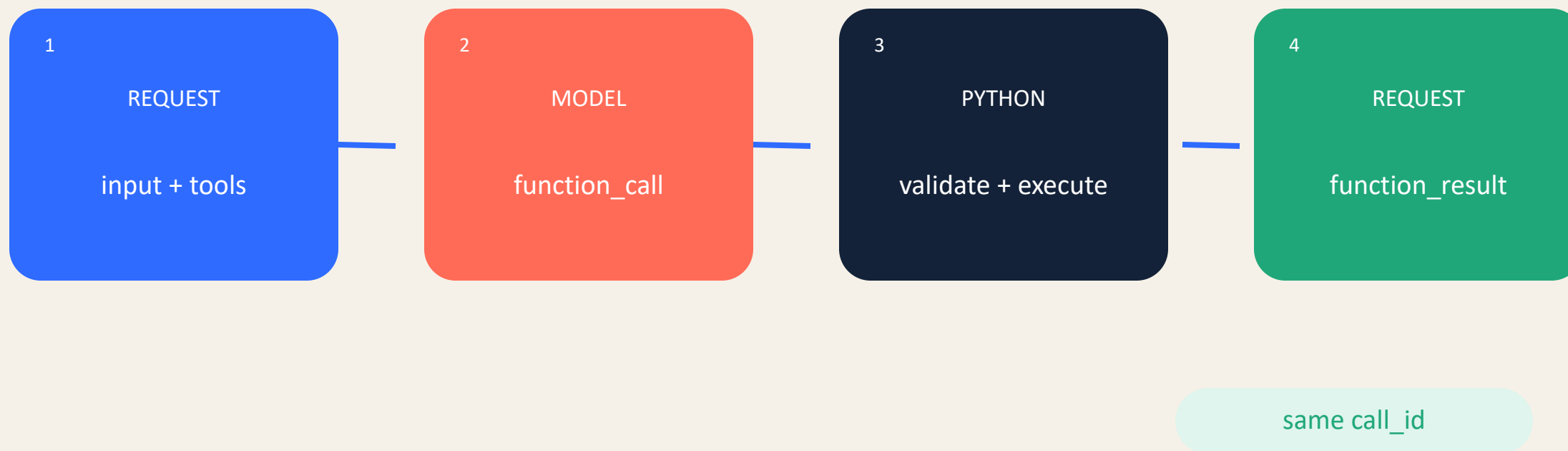
parameters

{ "type": "object", "properties": ... }

required

["path"]

Один корректный tool call — это четыре сообщения



practice/steps/04_correct_tool_calls.py

СЕЙЧАС

Добавить registry, schemas, safe paths, approval и timeout

Кто решает, что tool безопасен?

Tool

MODEL

выбирает name + args

SCHEMA

проверяет форму

REGISTRY

разрешает только известные tools

POLICY

safe path · approval · timeout

FUNCTION

делает side effect

Весь контекст



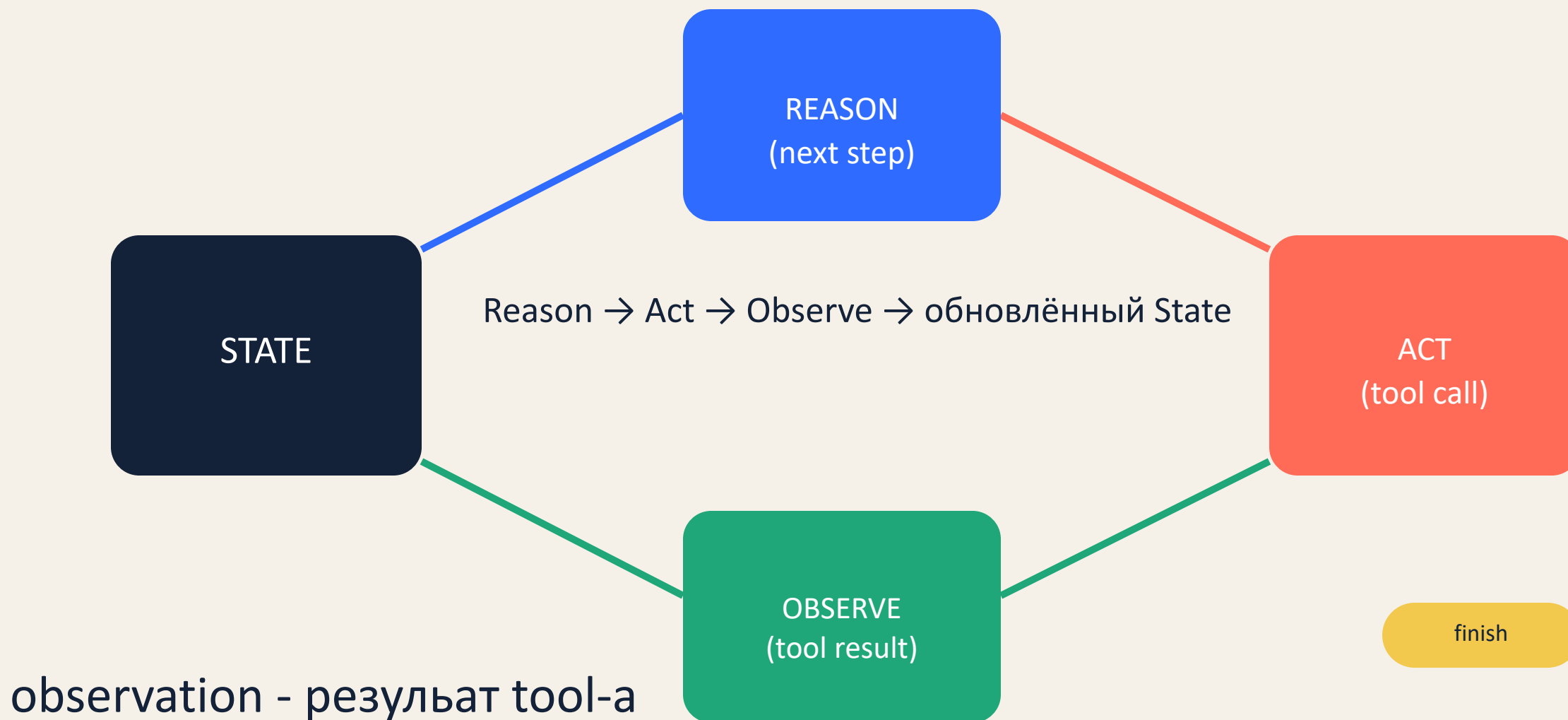
Число шагов заранее неизвестно

practice/steps/06_react_loop.py

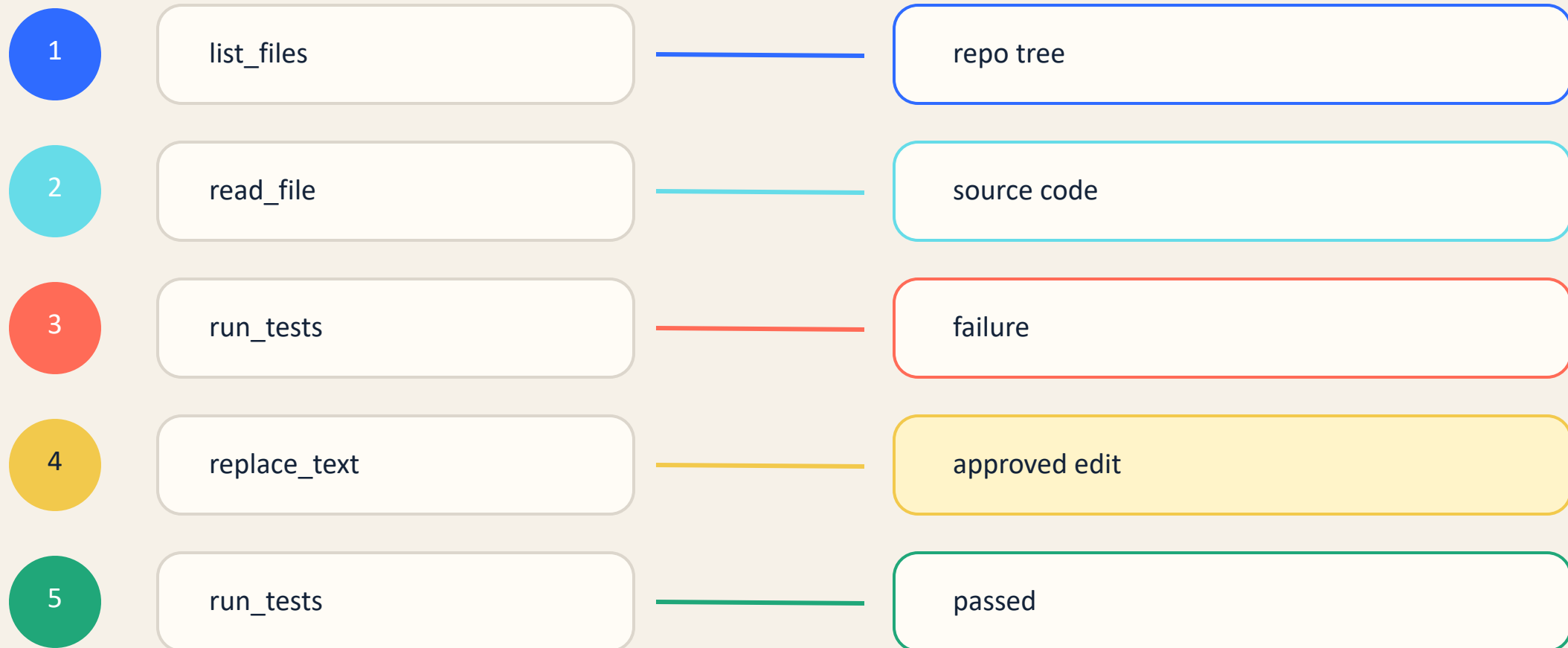
СЕЙЧАС

Цикл со state и stop conditions

ReAct: следующее действие выбирается после observation



ReAct-кодер: разные observations



ReAct гибкий, но платит за каждый следующий шаг



LATENCY

последовательные
API round-trips



COST

растущий
контекст



LOOPS

повторяет
одни calls



MYOPIA

видит локальный
следующий шаг

Лекарство: limits · state compression · planner · verifier

CoT, ReAct и Plan-and-Execute отвечают на разные вопросы

CoT

Как рассуждать?

reason → answer

нет внешних действий

ReAct

Что делать дальше?

reason ↔ act ↔ observe

адаптивный local loop

Plan-and-Execute

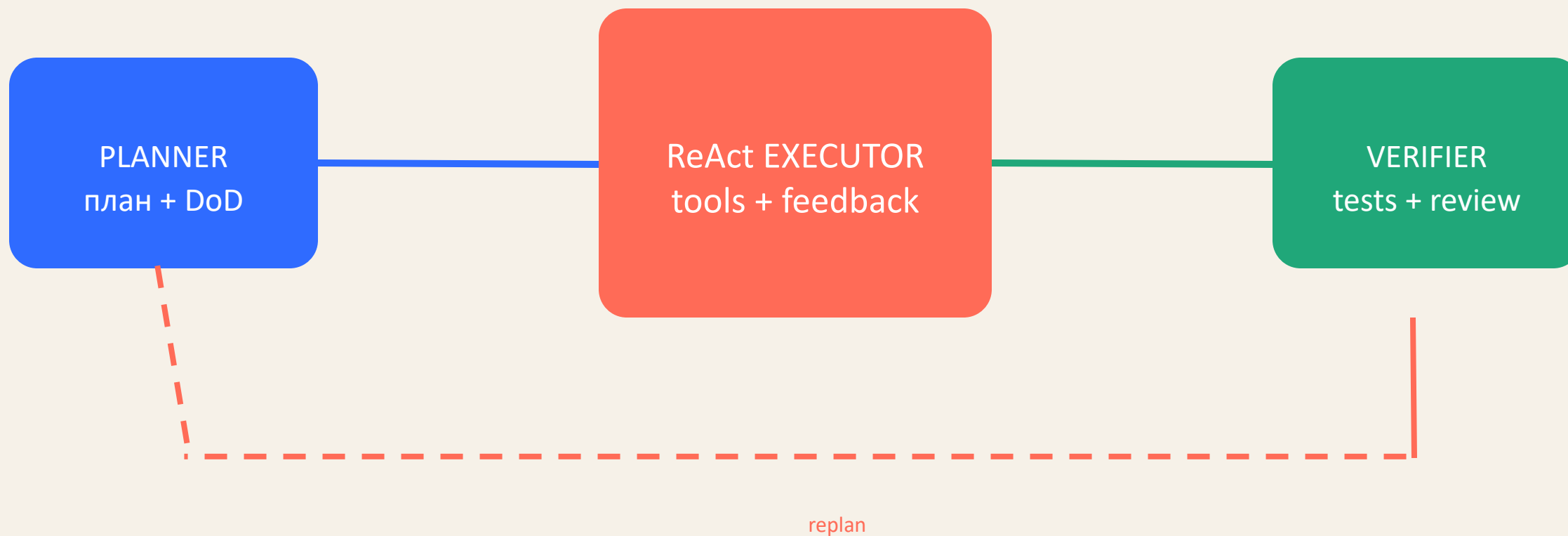
Как пройти весь путь?

plan → execute → replan

глобальная декомпозиция

На практике архитектуры смешивают

Глобальный план + локальная адаптивность + независимая проверка

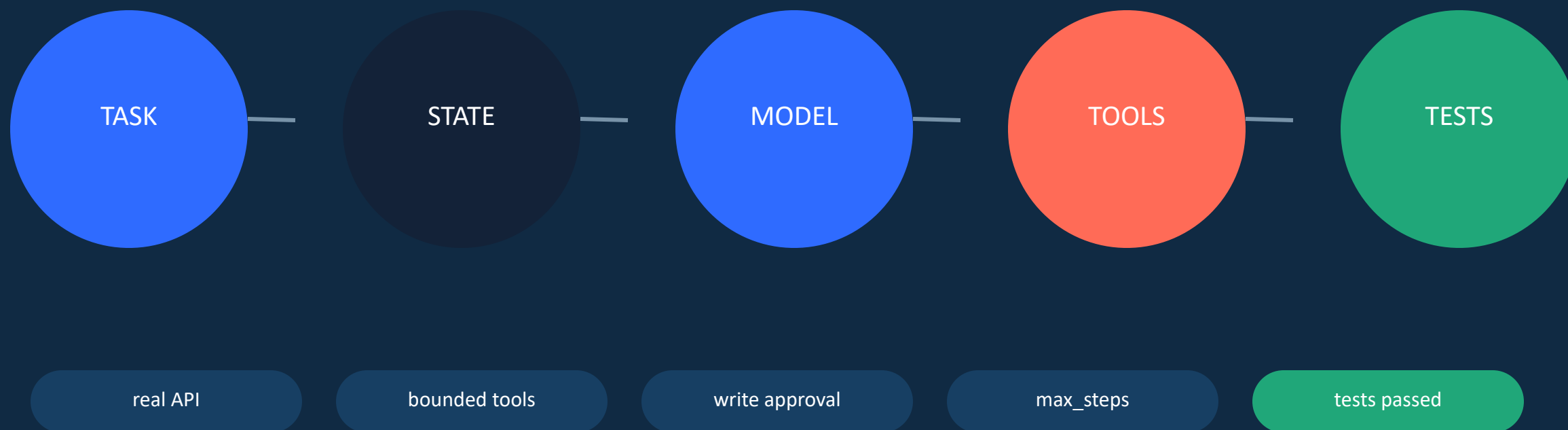


practice/steps/07_final_coding_agent.py

СЕЙЧАС

Собрать coder: read → test → approval → edit → retest → finish

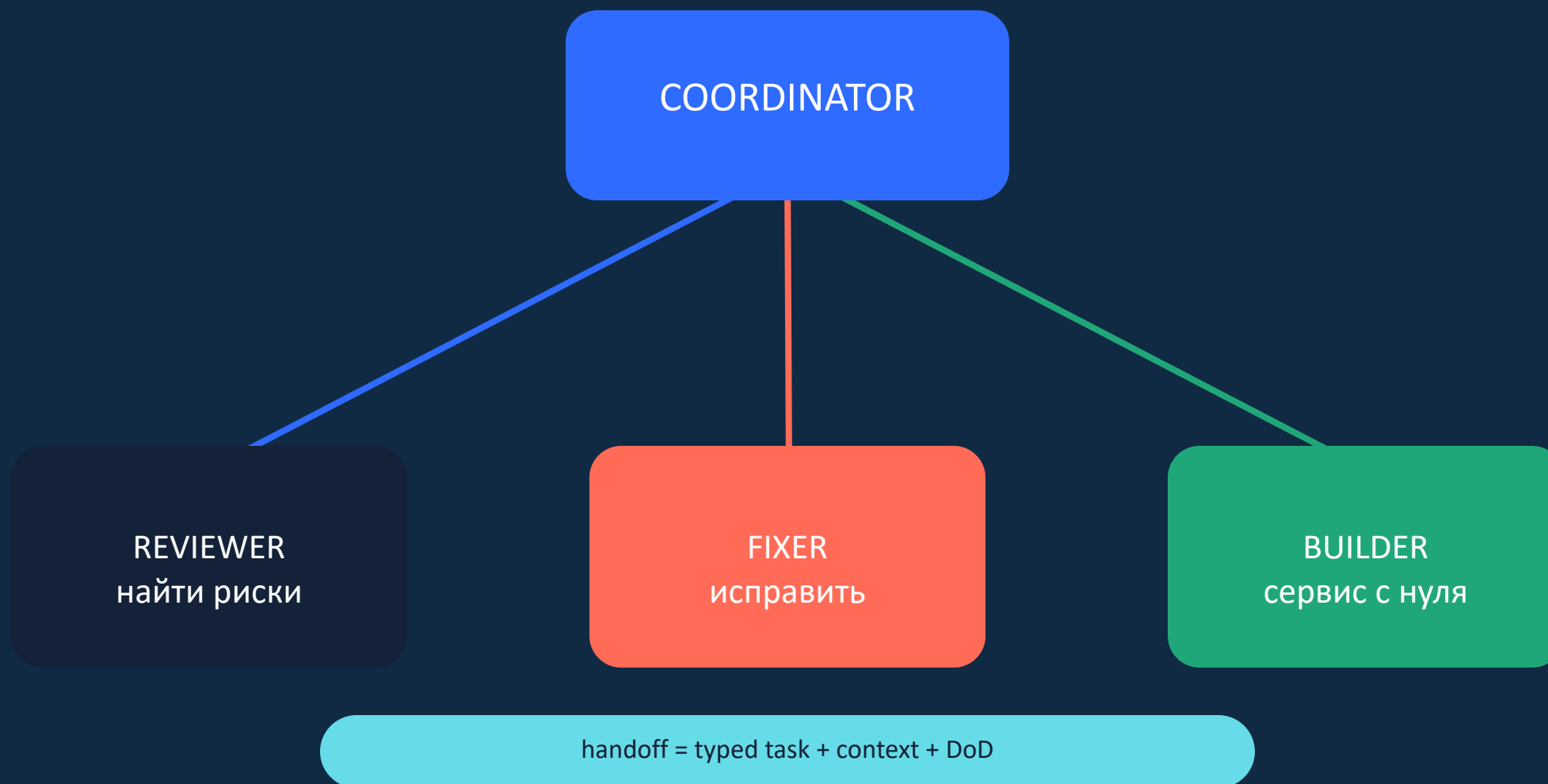
Финальный coder: модель решает, harness ограничивает



DONE = изменения применены + проверка прошла + trace сохранён

Если ролей много — появляется оркестрация

ЕСЛИ ЕСТЬ ВРЕМЯ → [practice/steps/08_optional_multiagent.py](#)



Prompt задаёт поведение; context даёт рабочую реальность

PROMPT ENGINEERING



роль · правила · формат
критерий остановки

CONTEXT ENGINEERING



файлы · тесты · history
observations · memory

ЕСЛИ ЕСТЬ ВРЕМЯ → [practice/steps/09_optional_context_engineering.py](#)

В главных ролях:

1

ReAct · понятное объяснение

<https://habr.com/ru/articles/882432/>

2

ReAct · original paper

<https://arxiv.org/abs/2210.03629>

3

ReAct vs Plan-and-Execute vs CoT

<https://chatme.ai/blog/arhitekturnye-patterny-ii-agentov-react-plan-and-e/>

4

Gemini API specification

<https://ai.google.dev/api>

5

Gemini function calling

<https://ai.google.dev/gemini-api/docs/function-calling>